

IA PARA DEVS

FASE 3 | AULA 04 -

MULTI-AGENT

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS.....	28
O QUE VOCÊ VIU NESTA AULA?	29
REFERÊNCIAS	30

EMBA

O QUE VEM POR AÍ?

Nesta aula, exploraremos sistemas multiagente usando LangGraph, para construir arquiteturas de IA que combinam múltiplos agentes, cada um com sua tarefa específica, trabalhando coordenados. Começaremos com uma introdução aos sistemas multiagente e explicação sobre como o LangGraph nos permite implementar esses sistemas usando StateGraph, uma estrutura baseada em grafos que organiza agentes como nós interconectados, compartilhando estado e possibilitando a execução de fluxos condicionais.

Em seguida, implementaremos o padrão ReAct (Reasoning + Acting) na prática, criando agentes que alternam entre raciocínio explícito e execução de ações, mantendo um histórico rastreável de decisões e permitindo a coordenação entre agentes. Então, construiremos um sistema de análise de mercado financeiro usando três agentes especializados: Agente Pesquisador, responsável pela coleta de dados de mercado; Agente Analista, que processa informações e gera insights; e o Agente Relatório, que sintetiza dados em um relatório executivo.

O sistema usará o StateGraph do LangGraph para coordenação, o estado compartilhado para troca de dados, ferramentas externas para coleta de informações e o padrão ReAct para raciocínio transparente. Veremos como implementar a coordenação entre agentes, tratar falhas, paralelismo controlado e persistência de estado, conceitos essenciais para sistemas multiagente em produção.

HANDS ON

Nesta videoaula vamos construir um sistema multiagente real usando o LangGraph para análise de mercado financeiro. Para isso, usaremos três agentes especializados, que trabalharão coordenados usando o StateGraph e opadrão ReAct. Na próxima seção, você acompanhará o passo a passo para a instalação e construção do sistema.



SAIBA MAIS

1. Instalação e Imports

```
# Instalação necessária

# pip install langgraph langchain langchain-openai python-
dotenv yfinance requests

import os

from typing import Dict, List, Any, TypedDict, Annotated

from datetime import datetime, timedelta

import yfinance as yf

import requests

from langchain_core.messages import HumanMessage,
AIMessage, SystemMessage

from langchain_core.tools import tool

from langchain_openai import ChatOpenAI

from langgraph.graph import StateGraph, END, START

from langgraph.prebuilt import ToolNode, tools_condition

from langgraph.checkpoint.memory import MemorySaver

# Configuração da API Key

os.environ["OPENAI_API_KEY"] = "sua_api_key_aqui"
```

Código-fonte 1 – Instalação e Imports necessários
Fonte: elaborado pelo autor (2025)

2. DEFININDO O ESTADO COMPARTILHADO

O estado é compartilhado entre todos os agentes e mantém o contexto completo da análise. Isto é fundamental para que a sequência das ações realizadas pelos agentes atinja o objetivo final.

```
class EstadoAnalise(TypedDict):  
    """Estado compartilhado entre todos os agentes"""  
  
    ticker: str  
  
    dados_mercado: Dict[str, Any]  
  
    analise_tecnica: str  
  
    insights_analista: str  
  
    relatorio_final: str  
  
    etapa_atual: str  
  
    historico_decisoos: List[str]  
  
    erro: str  
  
def criar_estado_inicial(ticker: str) -> EstadoAnalise:  
    """Cria estado inicial para análise"""  
  
    return EstadoAnalise(  
        ticker=ticker,  
        dados_mercado={},  
        analise_tecnica="",  
        insights_analista="",  
        relatorio_final="",  
        etapa_atual="inicio",  
        historico_decisoos=[],  
        erro=""  
    )
```

Código-fonte 2 – Definindo o estado compartilhado
Fonte: elaborado pelo autor (2025)

3. FERRAMENTAS PARA COLETA DE DADOS

```
@tool

def obter_dados_acao(ticker: str) -> Dict[str, Any]:
    """
        Coleta dados históricos e informações da ação.

        Args:
            ticker: Símbolo da ação (ex: AAPL, GOOGL)

        Returns:
            Dict com preços, volume, indicadores técnicos
    """
    try:
        acao = yf.Ticker(ticker)

        # Dados históricos (30 dias)
        hist = acao.history(period="30d")

        # Informações da empresa
        info = acao.info

        # Calculando métricas
        preco_atual = hist['Close'].iloc[-1]
        preco_30d_atras = hist['Close'].iloc[0]

        variacao_30d = ((preco_atual - preco_30d_atras) /
preco_30d_atras) * 100
```

```
volume_medio = hist['Volume'].mean()

    volatilidade = hist['Close'].pct_change().std() *
100

    return {

        "ticker": ticker,

        "preco_atual": round(preco_atual, 2),

        "variacao_30d": round(variacao_30d, 2),

        "volume_medio": int(volume_medio),

        "volatilidade": round(volatilidade, 2),

        "market_cap": info.get("marketCap", "N/A"),

        "setor": info.get("sector", "N/A"),

        "timestamp": datetime.now().isoformat()

    }

except Exception as e:

    return {"erro": f"Erro ao coletar dados: {str(e)}"}

@tool

def obter_noticias_mercado(ticker: str) -> List[Dict[str,
str]]:

    """

    Simula coleta de notícias relacionadas ao ticker.
```

Código-fonte 3 – Ferramentas para coleta de dados

Fonte: elaborado pelo autor (2025)

Em produção, usaríamos uma API “real”, como a [NewsAPI](#) ou o [Alpha Vantage](#).

4. SIMULAÇÃO DE NOTÍCIAS PARA DEMONSTRAÇÃO

```
noticias_simuladas = [  
    {  
        "titulo": f"Análise: {ticker} apresenta forte  
crescimento no Q4",  
        "resumo": "Empresa supera expectativas com  
inovação em IA",  
        "sentimento": "positivo"  
    },  
    {  
        "titulo": f"Mercado: Setor de {ticker}  
enfrenta volatilidade",  
        "resumo": "Fatores macroeconômicos afetam  
performance",  
        "sentimento": "neutro"  
    }  
]  
  
return noticias_simuladas
```

Código-fonte 4 – Simulação de notícias para demonstração
Fonte: elaborado pelo autor (2025)

5. AGENTE PESQUISADOR COM PADRÃO REACT

```
def agente_pesquisador(estado: EstadoAnalise) ->  
EstadoAnalise:  
    """  
    Agente responsável por coletar dados de mercado.  
  
    Implementa padrão ReAct: Thought → Action → Observation  
    """
```

```
print(f"\n    AGENTE      PESQUISADOR      -      Analisando
{estado['ticker']}")

# THOUGHT: Raciocínio explícito

pensamento = f"""

Thought: Preciso coletar dados completos sobre
{estado['ticker']}:

1. Dados históricos de preço e volume
2. Métricas fundamentais
3. Notícias recentes para análise de sentimento

Vou começar coletando dados da ação e depois buscar
notícias.

"""

print(pensamento)

estado["historico_decisoes"].append(f"Pesquisador:
{pensamento.strip()}")

try:

    # ACTION: Execução das ferramentas

    print("Action: Coletando dados da ação...")

    dados_acao = obter_dados_acao.invoke({"ticker":
estado["ticker"]})

    print("Action: Coletando notícias do mercado...")

    noticias =
obter_noticias_mercado.invoke({"ticker": estado["ticker"]})

# OBSERVATION: Processamento dos resultados
```

```
if "erro" in dados_acao:

    estado["erro"] = dados_acao["erro"]

    estado["etapa_atual"] = "erro"

    return estado


observacao = f"""

Observation: Dados coletados com sucesso:

- Preço atual: ${dados_acao['preco_atual']}
- Variação 30d: {dados_acao['variacao_30d']}%
- Volatilidade: {dados_acao['volatilidade']}%
- {len(noticias)} notícias analisadas

Dados prontos para análise técnica.

"""

print(observacao)

# Consolidando dados coletados

estado["dados_mercado"] = {

    "acao": dados_acao,

    "noticias": noticias,

    "timestamp_coleta": datetime.now().isoformat()

}

estado["etapa_atual"] = "dados_coletados"

estado["historico_decisoes"].append(f"Pesquisador:
{observacao.strip()}")
```

```
print(" Pesquisa concluída - Transferindo para Analista")

except Exception as e:

    estado["erro"] = f"Erro no agente pesquisador:
{str(e)}"

    estado["etapa_atual"] = "erro"

return estado
```

Código-fonte 5 – Agente pesquisador com padrão React
Fonte: elaborado pelo autor (2025)

6. AGENTE ANALISTA COM IA

```
def agente_analista(estado: EstadoAnalise) ->
EstadoAnalise:

    """
    Agente que processa dados e gera insights usando LLM.
    Implementa raciocínio avançado com LangChain.
    """

    print(f"\n AGENTE ANALISTA - Processando dados de
{estado['ticker']}")

    if not estado["dados_mercado"]:

        estado["erro"] = "Dados de mercado não
disponíveis"

        estado["etapa_atual"] = "erro"

        return estado
```

```
try:

    llm = ChatOpenAI(model="gpt-4o-mini",
temperature=0.3)

    # THOUGHT: Estruturando análise

    pensamento = ""

    Thought: Vou analisar os dados coletados em 3
dimensões:

    1. Análise técnica (preço, volume, volatilidade)
    2. Análise fundamentalista (market cap, setor)
    3. Análise de sentimento (notícias)

    Preciso gerar insights acionáveis para o relatório
final.

    """
    print(pensamento)

    estado["historico_decisoes"].append(f"Analista:
{pensamento.strip()}")

    # ACTION: Análise com LLM

    dados = estado["dados_mercado"]["acao"]

    noticias = estado["dados_mercado"]["noticias"]

    prompt_analise = f"""

    Você é um analista financeiro experiente. Analise
os dados abaixo e forneça insights:

    DADOS DA AÇÃO {estado['ticker']}:

    - Preço atual: ${dados['preco_atual']}
```

```
- Variação 30 dias: {dados['variacao_30d']}%
- Volatilidade: {dados['volatilidade']}%
- Market Cap: {dados['market_cap']}
- Setor: {dados['setor']}

NOTÍCIAS RECENTES:

{chr(10).join([f"- {n['titulo']} (Sentimento:
{n['sentimento']})" for n in noticias])}

Forneça uma análise estruturada com:

1. Análise Técnica (tendência, volatilidade,
volume)

2. Análise Fundamentalista (valorização, setor)

3. Análise de Sentimento (notícias, momentum)

4. Recomendação (Compra/Venda/Manter) com
justificativa

5. Riscos principais identificados

Seja conciso mas técnico. Use dados quantitativos
quando possível.

"""

print("Action: Executando análise com LLM...")

mensagens = [

    SystemMessage(content="Você é um analista financeiro
experiente e objetivo."),

    HumanMessage(content=prompt_analise)

]

resposta = llm.invoke(mensagens)
```

```
# OBSERVATION: Processando insights

observacao = f"""

Observation: Análise completa gerada:

- Avaliou {len(dados)} métricas técnicas
- Processou {len(noticias)} fontes de notícias
- Gerou recomendação fundamentada
- Identificou riscos principais

Insights prontos para relatório executivo.

"""

print(observacao)

estado["insights_analista"] = resposta.content
estado["etapa_atual"] = "analise_concluida"
estado["historico_decisoes"].append(f"Analista:
{observacao.strip()}")

print(" Análise concluída - Transferindo para Gerador de
Relatório")

except Exception as e:

    estado["erro"] = f"Erro no agente analista:
{str(e)}"

    estado["etapa_atual"] = "erro"

return estado
```

Código-fonte 6 – Agente analista com IA

Fonte: elaborado pelo autor (2025)

7. AGENTE GERADOR DE RELATÓRIO

```
def agente_relatorio(estado: EstadoAnalise) -> EstadoAnalise:
    """
    Agente que sintetiza informações em relatório executivo.
    """
    print(f"\n AGENTE RELATÓRIO - Criando relatório de {estado['ticker']}")

    if not estado["insights_analista"]:
        estado["erro"] = "Insights do analista não disponíveis"
        estado["etapa_atual"] = "erro"
        return estado

    try:
        llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.1)

        # THOUGHT: Estruturando relatório
        pensamento = """
        Thought: Vou criar um relatório executivo profissional que:
        1. Resume dados-chave de forma visual
        2. Apresenta recomendação clara
        3. Justifica com dados quantitativos
        """
```



```
4. Identifica próximos passos

O relatório deve ser acionável para tomada de
decisão.

"""

print(pensamento)

estado["historico_decisoos"].append(f"Relatório:
{pensamento.strip()}")

# ACTION: Geração do relatório

dados = estado["dados_mercado"]["acao"]

prompt_relatorio = f"""

Crie um RELATÓRIO EXECUTIVO profissional para a
ação {estado['ticker']}.

DADOS DISPONÍVEIS:

{estado['insights_analista']}

DADOS QUANTITATIVOS:

- Preço: ${dados['preco_atual']} | Variação 30d:
{dados['variacao_30d']}%

- Volatilidade: {dados['volatilidade']}% | Setor:
{dados['setor']}

FORMATO DO RELATÓRIO:

RELATÓRIO EXECUTIVO - {estado['ticker']}

{datetime.now().strftime("%d/%m/%Y %H:%M")}

RECOMENDAÇÃO: [COMPRA/VENDA/MANTER]
```

```
RESUMO EXECUTIVO:

[Parágrafo conciso com principal insight]

MÉTRICAS-CHAVE:

• Preço atual: ${dados['preco_atual']}
• Performance 30d: {dados['variacao_30d']}%
• Volatilidade: {dados['volatilidade']}%
• Status: [Resumo técnico]

ANÁLISE RISCO-RETORNO:

[Avaliação equilibrada]

PRÓXIMOS PASSOS:

[1-3 ações recomendadas]

Use emojis e formatação para facilitar leitura
executiva.

"""

print("Action: Gerando relatório executivo...")

mensagens = [

    SystemMessage(content="Você cria relatórios executivos
claros e acionáveis."),

    HumanMessage(content=prompt_relatorio)

]
```

```
resposta = llm.invoke(mensagens)

# OBSERVATION: Finalizando
observacao = """
Observation: Relatório executivo criado com
sucesso:

- Formato otimizado para tomada de decisão
- Recomendação clara baseada em dados
- Métricas-chave destacadas
- Próximos passos definidos

Sistema multiagente concluído com sucesso.
"""
print(observacao)

estado["relatorio_final"] = resposta.content
estado["etapa_atual"] = "concluido"

estado["historico_decisoes"].append(f"Relatório:
{observacao.strip()}")

print(" Relatório concluído - Sistema finalizado")

except Exception as e:

    estado["erro"] = f"Erro no agente de relatório:
{str(e)}"

    estado["etapa_atual"] = "erro"
```

```
return estado
```

Código-fonte 7 – Agente gerador de relatório
Fonte: elaborado pelo autor (2025)

8. CONSTRUINDO O STATEGRAPH

```
def criar_sistema_multiagente():  
    """  
    Cria o sistema multiagente usando StateGraph do  
    LangGraph.  
    """  
  
    # Definindo condições de roteamento  
    def deve_continuar(estado: EstadoAnalise) -> str:  
        """Determina próximo agente baseado no estado  
        atual"""  
        if estado["etapa_atual"] == "erro":  
            return "fim"  
        elif estado["etapa_atual"] == "inicio":  
            return "pesquisador"  
        elif estado["etapa_atual"] == "dados_coletados":  
            return "analista"  
        elif estado["etapa_atual"] == "analise_concluida":  
            return "relatorio"  
        elif estado["etapa_atual"] == "concluido":  
            return "fim"  
        else:  
            return "fim"
```

```
# Criando o grafo

workflow = StateGraph(EstadoAnalise)

# Adicionando agentes como nós

workflow.add_node("pesquisador", agente_pesquisador)
workflow.add_node("analista", agente_analista)
workflow.add_node("relatorio", agente_relatorio)

# Definindo fluxo condicional

workflow.add_conditional_edges(
    START,
    deve_continuar,
    {
        "pesquisador": "pesquisador",
        "fim": END
    }
)

workflow.add_conditional_edges(
    "pesquisador",
    deve_continuar,
    {
        "analista": "analista",
        "fim": END
    }
)
```

```
)

workflow.add_conditional_edges(
    "analista",
    deve_continuar,
    {
        "relatorio": "relatorio",
    "fim": END
    }
)

workflow.add_conditional_edges(
    "relatorio",
    deve_continuar,
    {
        "fim": END
    }
)

# Compilando com persistência de estado
memoria = MemorySaver()

app = workflow.compile(checkpointer=memoria)

return app
```

Código-fonte 8 – Construindo o Stategraph
Fonte: elaborado pelo autor (2025)

9. EXECUTANDO O SISTEMA

```
def executar_analise_mercado(ticker: str):  
    """  
    Executa análise completa de mercado com sistema  
multiagente.  
    """  
  
    print(f"❑ INICIANDO ANÁLISE MULTIAGENTE - {ticker}")  
    print("=" * 60)  
  
    # Criando sistema e estado inicial  
    sistema = criar_sistema_multiagente()  
    estado_inicial = criar_estado_inicial(ticker)  
  
    # Configuração para persistência  
    config = {"configurable": {"thread_id":  
f"analise_{ticker}_{datetime.now().isoformat()}"}}  
  
    try:  
        # Executando fluxo completo  
        resultado = sistema.invoke(estado_inicial,  
config=config)  
  
        print("\n" + "=" * 60)  
        print(" ANÁLISE MULTIAGENTE CONCLUÍDA")  
        print("=" * 60)  
  
        if resultado["erro"]:
```

```
print(f" Erro: {resultado['erro']}")

    return None

    # Exibindo relatório final
print("\n RELATÓRIO EXECUTIVO:")
print("-" * 40)

    print(resultado["relatorio_final"])

    # Exibindo histórico de decisões
print("\n HISTÓRICO DE DECISÕES:")
print("-" * 40)

    for i, decisao in enumerate(resultado["historico_decisoes"], 1):
        print(f"{i}. {decisao}")

    return resultado

except Exception as e:
    print(f" Erro na execução: {str(e)}")
    return None

# Exemplo de uso

if __name__ == "__main__":
    # Testando o sistema

        resultado = executar_analise_mercado("AAPL")

    if resultado:
```



```
print(f"\n Análise de {resultado['ticker']} concluída com  
sucesso!")  
  
print(f" Estado final: {resultado['etapa_atual']}")
```

Código-fonte 9 – Executando o sistema
Fonte: elaborado pelo autor (2025)

SISTEMAS MULTIAGENTE

Sistemas multiagente são uma abordagem da Inteligência Artificial que modela o comportamento coletivo, por meio de agentes autônomos que operam em cooperação. Cada agente é responsável por uma parte do problema, cuja resolução emerge da interação entre esses agentes. Isso os torna particularmente úteis em ambientes dinâmicos, distribuídos e complexos, onde uma entidade central não daria conta de gerenciar todas as decisões.

Inspirados por fenômenos naturais (como, por exemplo, colônias de formigas ou bandos de pássaros) e por estruturas sociais humanas (como equipes multidisciplinares), sistemas construídos com este paradigma podem trazer ganhos de **modularidade, resiliência, paralelismo e inteligência coletiva**.

CONCEITOS FUNDAMENTAIS

1. AGENTE

Um **agente** é uma entidade autônoma que percebe seu ambiente por meio de sensores (ou entradas) e age sobre ele através de atuadores (ou saídas), com base em objetivos definidos.

Em Inteligência Artificial, os agentes podem ser simples, como bots reativos, ou complexos, como agentes baseados em modelos de linguagem, que usam memória, ferramentas e aprendizado.

2. MULTIAGENTE

Um **sistema multiagente (ou “MAS”)** é composto por múltiplos agentes que interagem entre si. Cada agente possui objetivos próprios ou compartilhados, podendo cooperar, competir ou operar de forma independente. As decisões são distribuídas e, muitas vezes, coordenadas dinamicamente com base em regras ou protocolos.

Exemplos clássicos incluem:

- Trânsito inteligente (veículos e semáforos como agentes);
- Robótica em enxame (drones autônomos colaborando);
- Mercados financeiros simulados (agentes compradores e vendedores).

3. PADRÃO REACT

O padrão **ReAct (Reasoning + Acting)** é uma arquitetura emergente para construção de agentes baseados em LLMs. Ele combina dois elementos fundamentais:

- **Raciocínio explícito** (“pensar em voz alta”), no qual o modelo elabora seu raciocínio antes de decidir a próxima ação;
- **Execução de ações**, no qual o agente invoca ferramentas ou funções com base em seu raciocínio.

O ciclo segue a lógica:

Thought: "Preciso entender o que o usuário quer."

Action: "Consultar API de contexto."

Observation: "A resposta da API foi XYZ."

Thought: "Com isso, posso formular uma resposta clara."

Esse padrão melhora a interpretabilidade e permite estruturar o comportamento dos agentes em ambientes mais complexos, mantendo rastreabilidade e lógica.

4. LANGGRAPH

Como visto, o LangGraph é uma biblioteca que permite construir arquiteturas de IA baseadas em máquinas de estados e fluxos dirigidos por grafos. Neste contexto, ele facilita a composição de agentes que:

- São modelados como nós no grafo;
- Executam lógica condicional e ciclos;
- Compartilham contexto de forma controlada;
- Operam de forma modular, rastreável e extensível.

O LangGraph oferece mais controle do que pipelines lineares ou simples encadeamentos de prompts, sendo ideal para workflows com bifurcações, paralelismo, falhas e validações.

MERCADO, CASES E TENDÊNCIAS

As **principais empresas de tecnologia** estão implementando sistemas **multiagente** como estratégia para escalar a Inteligência Artificial empresarial, coordenar modelos especializados e automatizar fluxos complexos de decisão.

1. MICROSOFT - AUTOGEN: ORQUESTRAÇÃO DE LLMS COM ROI COMPROVADO

A Microsoft desenvolveu o [AutoGen](#). Esse framework permite que múltiplos agentes LLM conversem e colaborem em tarefas complexas.

IMPLEMENTAÇÃO TÉCNICA:

- **Arquitetura:** agentes baseados em diferentes LLMs (GPT-4, Claude, Llama);
- **Coordenação:** protocolo de conversação assíncrona com validação cruzada;
- **Ferramentas:** integração com Code Interpreter, APIs externas e bases de conhecimento.

CASE REAL - DESENVOLVIMENTO DE SOFTWARE:

- **Agente Programador:** escreve código baseado em especificações;
- **Agente Revisor:** analisa “bugs” e padrões de código;
- **Agente Testador:** cria e executa testes automatizados;
- **Agente Documentador:** gera documentação técnica.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos em profundidade o conceito de **sistemas multiagente** aplicados à inteligência artificial moderna. Vimos como essa abordagem permite estruturar soluções complexas a partir da **cooperação entre agentes autônomos**, cada um com um papel bem definido dentro de um ecossistema coordenado.

Discutimos como a **modularidade**, princípio central dos sistemas multiagente, favorece não apenas a clareza do fluxo lógico, mas também a **escalabilidade**, a **manutenção evolutiva** e a **reutilização de componentes** em diferentes contextos. Essa divisão de responsabilidades entre agentes especializados reduz a complexidade de desenvolvimento e facilita o isolamento e depuração de problemas.

Essa abordagem se mostra especialmente poderosa em cenários que exigem **raciocínio encadeado**, **decisões condicionais**, uso de **ferramentas externas** e interação contínua com ambientes dinâmicos, características presentes nas aplicações contemporâneas de IA, como copilotos, automação de fluxos de negócio e agentes conversacionais especializados.

REFERÊNCIAS

AGENTS for Amazon Bedrock. **Amazon**. [s.d.]. Disponível em: <https://docs.aws.amazon.com/bedrock/latest/userguide/agents.html>. Acesso em: 01 ago. 2025.

AUTOGEN Framework. **Microsoft Research**. [s.d.]. Disponível em: <https://microsoft.github.io/autogen/>. Acesso em: 01 ago. 2025.

CLAUDE 3.5 Sonnet. **Anthropic**. 2024. Disponível em: <https://www.anthropic.com/index/claude-3-5-sonnet>. Acesso em: 01 ago. 2025.

GEMINI. **Google DeepMind**. [s.d.]. Disponível em: <https://deepmind.google/technologies/gemini/>. Acesso em: 01 ago. 2025.

LANGGRAPH Documentation. **LangGraph**. [s.d.]. Disponível em: <https://www.langchain.com/langgraph>. Acesso em: 01 ago. 2025.

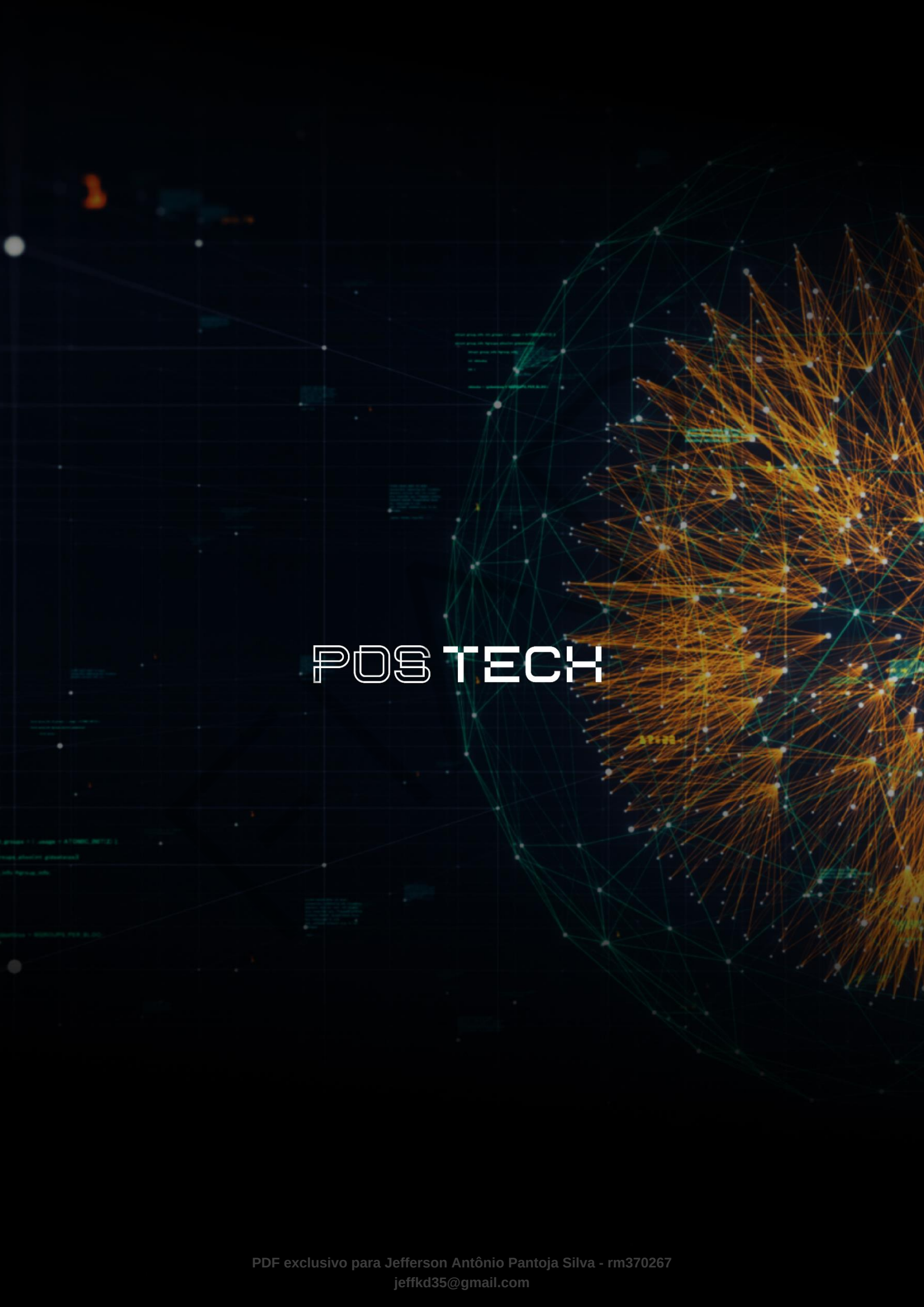
RUSSELL, S., NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4ª edição. Londres: Pearson, 2021.

SHUNYU, Y. et. AL. **ReAct: Synergizing Reasoning and Acting in Language Models**. 2022. Cornell University. Disponível em: <https://arxiv.org/abs/2210.03629>. Acesso em: 01 ago. 2025.

PALAVRAS-CHAVE

Multiagente. LangGraph. ReAct. LLMs. Coordenação de Agentes.

EMEND



POSTECH